COS 350 Lecture 6

struct

unix: who

case study: who

utmp file format, structure, fields

file descriptors – unbuffered files

localtime(), fields

The aggregate type in C is a `struct`

```
struct info {
    int age;
    char *name;
};

struct info bartinfo;

bartinfo.age = 12;
bartinfo.name = "Bart";

struct info lisainfo = {10, "Lisa"};
```

The aggregate type in C is a `struct`

```
struct info {
    int age;
    char *name;
};

struct info bartinfo;

bartinfo.age = 12;
bartinfo.name = "Bart";

struct info lisainfo = {10, "Lisa"};
```

We often use pointers to structures

```
struct info *person;

person = &bartinfo;
person->age = 13;
```

RULE

if have a struct
use . to get field

if have a pointer to a struct
use -> to get field

struct from sys/time.h

```
typedef long int time_t;
typedef long int suseconds_t;

struct timeval {
  time_t      tv_sec;          /* seconds */
  suseconds_t tv_usec;         /* and microseconds */
};
```

function:

```
int gettimeofday(struct timeval *tv, struct timezone *tz);
```

fills in the timeval struct (timezone is obsolete)
returns 0 for success and -1 for failure

example:

```
struct timeval tv;
gettimeofday(&tv, NULL);
printf("time is: %d", tv.tv_sec);
```

output:

time is: 1359489139

struct from time.h

```
struct tm {
  int tm_sec;        /* seconds */
  int tm_min;        /* minutes */
  int tm_hour;       /* hours */
  int tm_mday;       /* day of the month */
  int tm_mon;        /* month */
  int tm_year;       /* year */
  int tm_wday;       /* day of the week */
  int tm_yday;       /* day in the year */
  int tm_isdst;      /* daylight saving time */
};
```

<u>function:</u>

struct tm *localtime(time_t *timep);

breaks the time_t value down into: sec, min, hour, …
fills in a static structure with the values
returns a pointer to this structure or NULL on error

<u>example:</u>

```
struct tm *tmptr;
tmptr = localtime(&tv.tv_sec);
```

function:

```
char *ctime(time_t *timep);
```

converts the time_t value into a string of the form:

```
"Wed Jun 30 21:49:08 1993\n"
```

fills in a static string buffer

returns a pointer to the string or NULL on error

example:

```
printf("%s\n", ctime(&tv.tv_sec));
```

output:

```
Fri Jan 26 14:52:19 2018
```

```
unix> who
klevanets pts/0  2014-01-21 21:27 (cpe-67-253-21-55.maine.res.rr.com)
boothe    pts/1  2014-01-22 10:51 (boothe1.cs.usm.maine.edu)
boothe    tty7   2014-01-22 11:06
lienhardt pts/9  2014-01-22 11:22 (wasp.cs.usm.maine.edu)
```

user | terminal | login time | remote host

```
WHO(1)                User Commands                WHO(1)

NAME
       who - show who is logged on

SYNOPSIS
       who [OPTION]... [ FILE | ARG1 ARG2 ]

DESCRIPTION
       Print  information  about  users who are currently
       logged in.

       -a   same as -b -d -l -p -r -t -T -u
       -b    time of last system boot
       -d    print dead processes
       -l    print system login processes
```

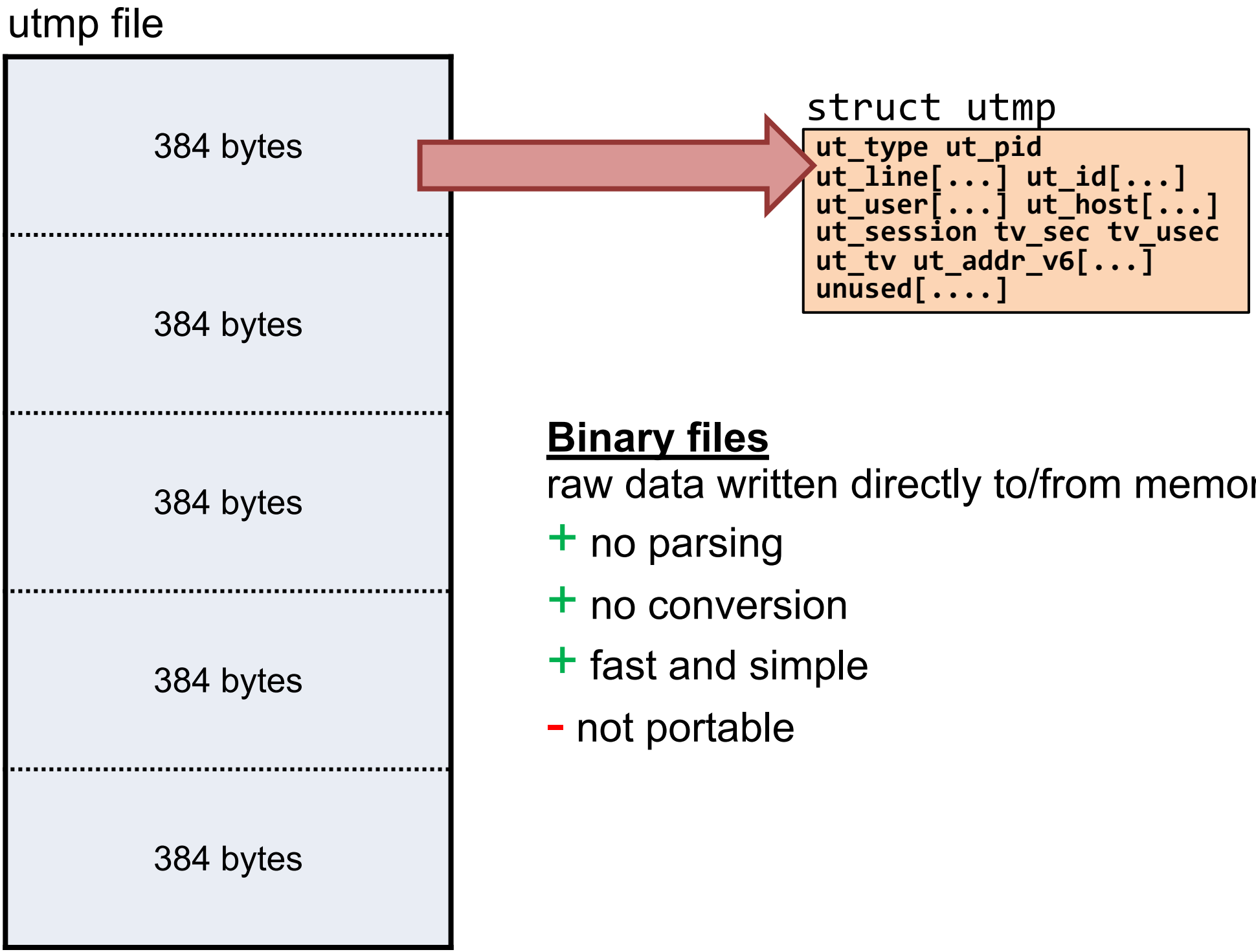
Case study: who

1. What does it does it do?
   List users logged into computer.
2. How?
   Linux maintains a binary file /var/run/utmp that contains login records

```
UTMP(5)                    Linux Programmer's Manual                    UTMP(5)

NAME
       utmp, wtmp - login records

SYNOPSIS
       #include <utmp.h>

DESCRIPTION
       The utmp file allows one to discover information about who is currently
       using the system.

       The  file  is  a  sequence  of  utmp structures, declared as follows in
       <utmp.h>

       struct utmp {
              short   ut_type;              /* Type of record */
```

```
struct utmp {
  short   ut_type;              /* Type of record */
  pid_t   ut_pid;               /* PID of login process */
  char    ut_line[UT_LINESIZE]; /* Device name of tty */
  char    ut_id[4];             /* Terminal name suffix */
  char    ut_user[UT_NAMESIZE]; /* Username */
  char    ut_host[UT_HOSTSIZE]; /* Hostname for remote login*/
  struct  exit_status ut_exit;  /* Exit status of a process */
  int32_t ut_session;           /* Session ID (getsid(2)) */
  struct {
    int32_t tv_sec;             /* Seconds */
    int32_t tv_usec;            /* Microseconds */
  } ut_tv;                      /* Time entry was made */
  int32_t ut_addr_v6[4];        /* Internet addr of remote host
  char __unused[20];            /* Reserved for future use */
};
```

sizeof(struct utmp) → 384 bytes



Binary files

raw data written directly to/from memory

+ no parsing
+ no conversion
+ fast and simple
- not portable

Raw un-buffered file operations (Unix system calls)

```
#include <fcntl.h>

int open(const char *pathname, int flags);

returns:
  a file descriptor (small integer)
  or -1  if  an  error occurred
```

O_RDONLY
O_WRONLY
O_RDWR

unistd.h

```
...
/* Standard file descriptors.  */
#define STDIN_FILENO   0  /* Standard input */
#define STDOUT_FILENO  1  /* Standard output */
#define STDERR_FILENO  2  /* Standard error output */
...
```

READ(2) Linux Programmer's Manual READ(2)

NAME

read - read from a file descriptor

SYNOPSIS

```
#include <unistd.h>

 size_t read(int fd, void *buf, size_t count);
```

DESCRIPTION

read() attempts to read up to count bytes from file descriptor fd into the buffer starting at buf.

RETURN VALUE

number of bytes read
0 indicates end of file
-1 is returned on error

CLOSE(2) Linux Programmer's Manual CLOSE(2)

NAME

close - close a file descriptor

SYNOPSIS

```
#include <unistd.h>

int close(int fd);
```

DESCRIPTION

close() closes a file descriptor, so that it no longer refers to any file and may be reused.

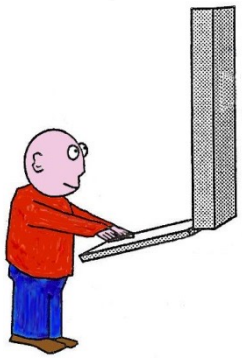
RETURN VALUE

0 on success
-1 on error

who1.c

```
#define INFOFILE  "/var/run/utmp"

int main(int ac, char **av)
{
  struct utmp utbuf;                    /* read info into here */
  int   utmpfd;                         /* read from this descriptor */

  if ( (utmpfd = open( INFOFILE, O_RDONLY )) == -1 ){
    fprintf(stderr,"%s: cannot open %s\n", *av, INFOFILE);
    exit(1);
  }

  while (read(utmpfd, &utbuf, sizeof(utbuf)) == sizeof(utbuf))
    show_info( &utbuf );

  close( utmpfd );
  return 0;                             /* program went ok */
}
```

```c
/*
 *  show info()
 *      displays the contents of the utmp struct
 *      in human readable form
 *      *note* these sizes should not be hardwired
 */
void show_info( struct utmp *utbufp )
{
  printf("%-8s", utbufp->ut_name);      /* the logname */
  printf(" ");                          /* a space  */
  printf("%-8s", utbufp->ut_line);      /* the tty  */
  printf(" ");                          /* a space  */
  printf("%10d", utbufp->ut_time);      /* login time */
  printf(" ");                          /* a space  */
  printf("(%s)", utbufp->ut_host);      /* the host */
  printf("\n");                         /* newline  */
}
```



who vs who1

Problem 1: non-relevant records

man utmp

```
/* Values for ut_type field */

#define EMPTY         0    /* Record does not contain valid info */
#define RUN_LVL       1    /* Change in system run-level (see init(8)) */
#define BOOT_TIME     2    /*Time of system boot (in ut_tv) */
#define NEW_TIME      3    /* Time after system clock change */
#define OLD_TIME      4    /* Time before system clock change */
#define INIT_PROCESS  5    /* Process spawned by init(8) */
#define LOGIN_PROCESS 6    /* Session leader process for user login */
#define USER_PROCESS  7    /* Normal process */
#define DEAD_PROCESS  8    /* Terminated process */
```

```
Void show_info( struct utmp *utbufp )
{
  if ( utbufp->ut_type != USER_PROCESS )
    return;

  printf("%-8s", utbufp->ut_name);      /* the lognam
  printf(" ");                          /* a space  *
  printf("%-12.12s", utbufp->ut_line);  /* the tty  *
  printf(" ");                          /* a space  *
```

Problem 2: time is just a raw number

function:

struct tm *localtime(time_t *timep);

breaks the time_t value down into: sec, min, hour, …
fills in a static structure with the values
returns a pointer to this structure or NULL on error

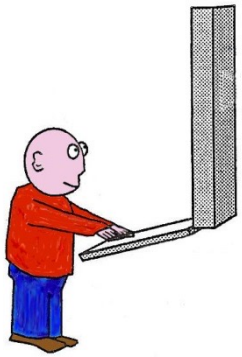
```
struct tm {
    int tm_sec;
    int tm_min;
    int tm_hour;
    int tm_mday;
    int tm_mon;
    int tm_year;
    int tm_wday;
    int tm_yday;
    int tm_isdst;
};
```

```
/*  displays time in a format fit for human consumption
 *  uses localtime to convert the timeval to
 *     year-month-day hour:min
 */
void showtime( time_t timeval ) {
  struct tm *tm;
  tm = localtime(&timeval);

  printf("%4d-%02d-%02d %02d:%02d",
         tm->tm_year + 1900,   // field holds: year-1900
         tm->tm_mon + 1,       // field holds: [0-11]
         tm->tm_mday,
         tm->tm_hour,
         tm->tm_min
        );
}
```

Problem 3: extra () if no remote host

```
unix> who
klevanets pts/0  2014-01-21 21:27 (cpe-67-253-21-55.maine.res.rr.com)
boothe    pts/1  2014-01-22 10:51 (boothe1.cs.usm.maine.edu)
boothe    tty7   2014-01-22 11:06 ()
lienhardt pts/9  2014-01-22 11:22 (wasp.cs.usm.maine.edu)
```

```
Void show_info( struct utmp *utbufp )
{
  if ( utbufp->ut_type != USER_PROCESS )
    return;

  printf("%-8.8s", utbufp->ut_name);   /* the logname  */
  printf(" ");                         /* a space  */
  printf("%-12.12s", utbufp->ut_line); /* the tty  */
  printf(" ");                         /* a space  */
  showtime( utbufp->ut_time );         /* display time */
  if ( utbufp->ut_host[0] != '\0' )
    printf(" (%s)", utbufp->ut_host);  /* the host */
  printf("\n");                        /* newline  */
}
```



who2

who3

does file buffering (but not that important for a small file)

read in book